DevOps Lifecycle and Automated Web Server Configuration using Ansible

Assignment Report

Aakanksha Sah 202301070148 Practical Btach-03

1. Abstract

This assignment presents a simple DevOps scenario in which a team needs to configure a web server consistently and reduce repetitive manual work. Ansible is selected as the configuration-management and automation tool. A static inventory identifies the target host, while a YAML playbook describes the desired configuration: install Nginx, ensure that its service is enabled and running, and deploy a sample HTML homepage. The report also describes the DevOps lifecycle, workflow, architecture, automation requirements, validation plan, and demonstration plan.

2. Problem Statement

Setting up a web server manually can lead to inconsistent configuration, missed steps, and repeated effort when environments need to be recreated. The objective is to define the configuration as code so that the same tasks can be applied in a repeatable and understandable way.

3. Objectives

- Select a real-world application scenario suitable for DevOps automation.
- Explain the lifecycle stages and workflow from planning through monitoring.
- Identify the major components in a basic Ansible architecture.
- Prepare an Ansible inventory file for the target host.
- Develop a playbook that installs and configures a web server and deploys a homepage.
- Define validation checks, screenshots, and a short demonstration plan.

4. Selected Real-World Scenario

Scenario: A small organization needs a basic internal website. A developer or system administrator must configure a Linux web server, make sure the service starts automatically, and publish the homepage. Ansible automates these tasks through a playbook rather than relying on a manual sequence of commands.

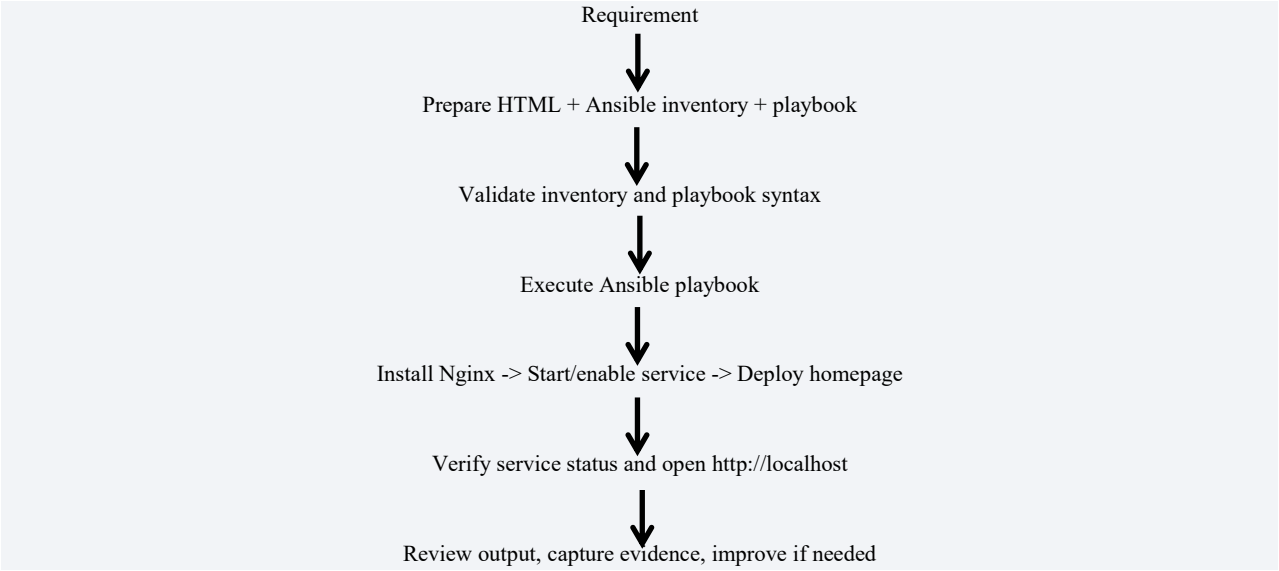
Scope: This educational demonstration uses a single Ubuntu/Debian-based host and Nginx. The supplied inventory uses Ansible's local connection so the playbook targets the same Linux environment where Ansible runs. A production setup could instead target a remote server over SSH.

5. DevOps Lifecycle Analysis

Lifecycle Stage	Application in This Project
Plan	Identify the requirement: provide a reliable web server and a simple homepage. Define success criteria: Nginx installed, service active, and homepage available over HTTP.
Develop	Write the HTML homepage and store the infrastructure configuration in version-controlled files. The Ansible playbook is the configuration-as-code artifact.

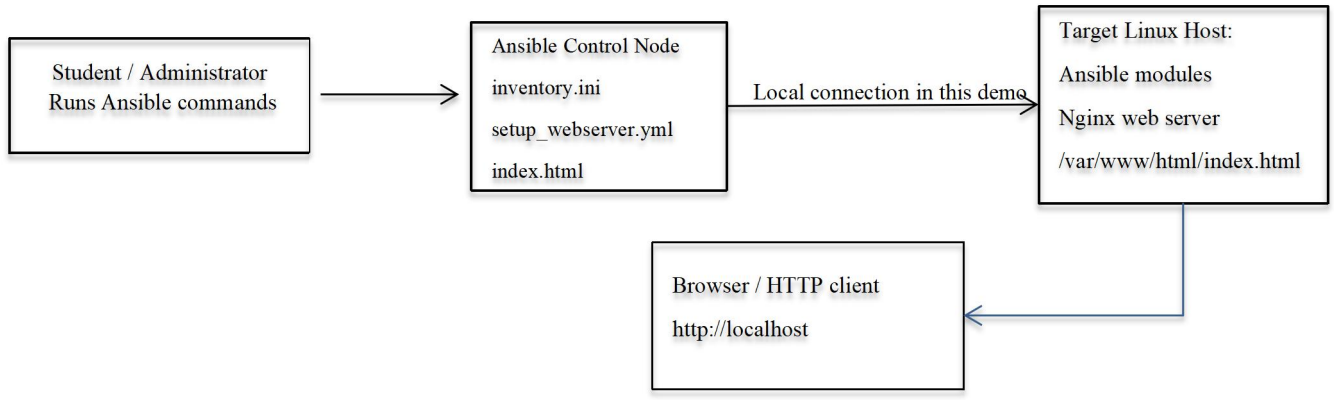
Build / Configure	Use an inventory to identify the target host and a YAML playbook to define the required configuration.
Test	Check Ansible syntax, validate the inventory, run the playbook, inspect the service status, and request the homepage with a browser or curl.
Release	Review the playbook and files, then make the approved version available for deployment. In a larger team, a Git repository and review process would be used.
Deploy	Run the Ansible playbook. It installs Nginx, starts and enables the service, and copies the HTML page to the web root.
Operate	Keep the web server configuration consistent by rerunning the playbook when required. Ansible modules generally check the desired state and avoid unnecessary changes.
Monitor / Feedback	Check service health and HTTP response, review execution output and logs, and use any failures to improve the playbook.

6. Proposed DevOps Workflow



The workflow is repeatable: configuration changes are made in the files, validated, and applied by rerunning the playbook. For a larger project, Git-based version control and an automated CI pipeline could run syntax checks and tests before deployment.

7. Architecture



Components: the control node runs Ansible; the inventory lists the target; the playbook describes tasks; Ansible modules perform operations; Nginx serves the HTML file; and a browser or HTTP client verifies the result. For remote management, the inventory would contain a server hostname or IP address and connection details, and Ansible would normally connect over SSH.

8. Tools and Automation Requirements

- Operating system: Ubuntu or another supported Debian-based Linux environment.
- Automation tool: Ansible.
- Web server: Nginx.
- Configuration formats: INI inventory, YAML playbook, and HTML webpage.
- Privileges: sudo/become access is needed to install packages and write to the web root.
- Network access: package repositories must be reachable when installing Nginx.
- Validation tools: ansible-inventory, ansible-playbook, systemctl, curl, and a web browser.

9. Ansible Inventory

Filename: inventory.ini. An inventory groups managed hosts and provides Ansible with a target for execution. This small demonstration targets localhost using a local connection.

```
[webservers]
localhost ansible_connection=local
```

Explanation: [webservers] defines a host group. The localhost entry identifies the local Linux machine, and ansible_connection=local tells Ansible not to use SSH for this demonstration. A remote deployment would use the actual host address and SSH configuration.

10. Ansible Playbook

Filename: setup_webserver.yml. The playbook describes the desired state and executes tasks in order. It uses privilege escalation because installing packages and writing the web root require administrative privileges.

```
---
- name: Configure a basic web server using Ansible
  hosts: webservers
  become: true
  gather_facts: true

  tasks:
    - name: Install Nginx web server
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true

    - name: Ensure Nginx is enabled and running
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true

    - name: Deploy the sample homepage
      ansible.builtin.copy:
```

```
    src: index.html
    dest: /var/www/html/index.html
    owner: root
    group: root
    mode: "0644"
```

10.1 Explanation of Tasks

- Install Nginx: the apt module ensures that the Nginx package is installed and refreshes package metadata.
- Ensure service is running: the service module starts Nginx and enables it to start at boot.
- Deploy homepage: the copy module transfers index.html to /var/www/html/index.html with readable file permissions.

11. Sample Webpage

Filename: index.html. This file is copied to the Nginx document root by the playbook. After successful execution, it should be available at http://localhost in the same Linux environment or through an appropriate Windows-to-WSL localhost route.

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>DevOps Ansible Demo</title>
 <style>
  body { font-family: Arial, sans-serif; margin: 4rem auto; max-width: 760px;
       padding: 0 1.5rem; background: #f4f7fb; color: #1f2937; }
  main { background: white; padding: 2rem; border-radius: 14px;
       box-shadow: 0 8px 24px rgba(0,0,0,.08); }
  h1 { color: #175cd3; }
 </style>
</head>
<body>
 <main>
  <h1>Web Server Configured with Ansible</h1>
  <p>This sample webpage is deployed automatically by an Ansible playbook.</p>
  <p>Scenario: repeatable web server configuration as part of a DevOps workflow.</p>
 </main>
</body>
</html>
```

12. Implementation and Execution Procedure

- Install and open a supported Ubuntu/Linux environment.
- Install Python 3 and Ansible using the distribution's package manager.
- Place inventory.ini, setup_webserver.yml, and index.html in the same project directory.
- Check the installation with: ansible --version
- Validate the inventory with: ansible-inventory -i inventory.ini --list
- Check playbook syntax with: ansible-playbook -i inventory.ini setup_webserver.yml --syntax-check
- Run the playbook with: ansible-playbook -i inventory.ini setup_webserver.yml --ask-become-pass
- Check Nginx with: systemctl status nginx --no-pager
- Check the page with: curl http://localhost and open http://localhost in a browser.

Note: The implementation commands are provided as a reproducible procedure. The playbook has not been executed as part of preparing this report; execution results must be collected from the student's own environment.

13. Testing and Validation Plan

Test	Method	Expected Result
Inventory parsing	Run ansible-inventory -i inventory.ini --list	The webservers group should contain localhost.
Syntax validation	Run ansible-playbook -i inventory.ini setup_webserver.yml --syntax-check	Ansible should report that the syntax is OK.
Package installation	Inspect playbook output and package state	Nginx should be installed.
Service health	Run systemctl status nginx --no-pager	Service should show active (running).
Homepage content	Run curl http://localhost or open the browser	The page should display the Ansible demo heading.
Repeatability	Run the playbook a second time	Already-correct settings should normally report no unnecessary changes.

14. Benefits

- Reduces repetitive manual configuration.
- Makes the desired configuration explicit and easy to review.
- Supports repeatable deployment across environments.
- Uses readable YAML tasks and a simple inventory.
- Can be extended to remote hosts, roles, version control, and CI pipelines.

15. Limitations and Future Improvements

- This is a single-host educational example, not a production high-availability architecture.
- The local connection avoids SSH setup but does not demonstrate remote-host connectivity.
- A production deployment should use a separate test environment, access controls, secure secrets handling, monitoring, and rollback planning.
- Future improvements could include Git version control, automated syntax checks in CI, Ansible roles, TLS/HTTPS, firewall configuration, and monitoring/alerting.

16. Conclusion

This project outlines how Ansible can automate basic web server configuration as part of a DevOps workflow. The inventory defines the target host, while the playbook describes the desired state of Nginx and the homepage. The proposed validation checks provide a simple way to verify deployment. Actual execution screenshots should be added after running the supplied files in a Linux environment.

17. References

1. Ansible Community Documentation, “Getting started with Ansible.” https://docs.ansible.com/projects/ansible/latest/getting_started/index.html

2. Ansible Community Documentation, “Building an inventory.” https://docs.ansible.com/projects/ansible/latest/getting_started/get_started_inventory.html

3. Ansible Community Documentation, “Creating a playbook.” https://ansible-community.github.io/package-doc-builds/getting_started/get_started_playbook.html